
MANUAL TÉCNICO

Estructura de datos y su uso en el programa

En el programa se dio el uso principal de 3 estructuras de datos para la búsqueda, eliminación, actualización y inserción de datos, después de todo tendremos que entender que a pesar de que pudo aplicarse un panorama simple de reducción a la aplicación de listas enlazadas y finalizar todo ahí. Pero el principal funcionamiento del programa se basa en árboles b con la implementación de montículos de Max heap y min heap para poder entender estructuras claves del programa, pero principalmente en la facilidad y escalabilidad que existe en los datos y su correcto almacenamiento.

Estructuras aplicadas

- **Árbol b+:** esta es la principal base del programa, la utilización de esta estructura se basa en el uso de su principal atractivo que radica en la utilización de la facilidad en el ordenamiento de datos porque nos simplificamos en meter datos a la estructura y en automático se estructura de tal manera que pueda ordenarse de manera ascendente y ordenada con una visualización gráfica sencilla de entender y con escalabilidad suficiente para nuestro programa y su búsqueda del uso de herramientas que en “automático” nos simplifiquen operaciones más complejas como la búsqueda e inserción de datos.

Esta herramienta puede ser vista en la sección de muestra de todos los datos, ya que mostrará simplemente se simplificará a mostrar una simple lista enlazada, pero con la facilidad de búsqueda de datos ordenada, pero su uso no finaliza ahí ya que será pieza clave para las demás funciones de búsqueda e inserción para poder sacar todo el provecho de esta herramienta.

- **Min heap:** nos permitirá mantener el nodo raíz siempre como el dato de menor valor numérico, permitiendo que con una sencilla búsqueda de dificultad $O(1)$ nos permitirá buscar dicho elemento en nuestra estructura de datos por medio de `peek()`. Esta aplicación se puede ver en la función 3 de nuestro menú al buscar al usuario con la menor pretensión salarial.

- Max heap: de manera similar nos permitirá encontrar el dato con mayor valor numérico, lo cual nos interesa por su aplicación de simplificar búsqueda de datos a $O(1)$ el cual se puede ver en la función de búsqueda de valor pretensión salarial, y con esto justifico el uso de montículos, debido a que a pesar de que en una lista enlazada podemos encontrar el dato de mayor o menor valor numérico con búsquedas y comparaciones for o con herramientas nativas del lenguaje de programación, lo que buscamos con la búsqueda de la estructura del Max y min heap es poder garantizar el menor o mayor dato para sencillamente buscarlo y/o eliminarlo con dificultad $O(1)$ sin importar la cantidad de datos que podamos encontrar almacenados en nuestro sistema con búsqueda de escalabilidad.

Diagrama de clases

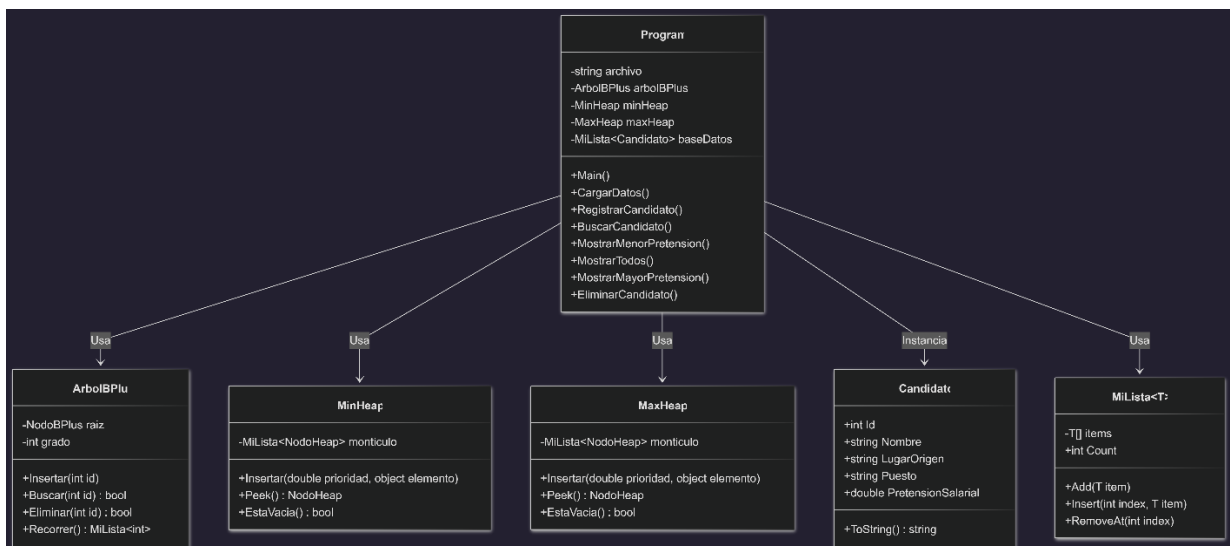


Diagrama de flujo

